

Stage 1 — Building the Data Layer

Technical Report on C++ Architecture, Implementation & Multi-Scale Benchmarking Analysis

Course: Big Data — B.Sc. Data Science and Engineering (ULPGC)	Date: October 2026
Team: TheScratchers	Branch: ft/c++-setup (C++17)
Author: Pablo Martínez Suárez	Repository: github.com/TheScratchers/stage_1

1. Executive Summary & Specification Compliance

This technical report documents the complete design, implementation, and multi-scale empirical evaluation of the C++ Data Layer for Stage 1 of the Big Data Search Engine. The system satisfies all core specification requirements and complies with the cross-language benchmark contract (shared/CONTRACT.md) across three main dimensions: raw datalake management, structured SQLite metadata storage, and triple inverted-index datamarts, backed by an automated 16-suite testing harness.

Key Architectural Accomplishments:

- **Tri-Layout Datalake:** Implements Time-Based (YYYYMMDD/HH/), Book-Based (ID/), and Batch-Based (batch_N/) hierarchies, featuring single-pass O(N) incremental detection and idempotent crash-recovery.
- **Three Distinct Inverted Index Datamarts:** Monolithic JSON, Alphabetical Hierarchical Filesystem, and a custom high-performance Binary Compact format (BIDX) with direct seek dictionary tables.
- **High-Throughput SQLite Metadata Mart:** Transaction-wrapped bulk ingestion sustaining over 580,000 rows/second with B-tree index query acceleration.
- **RAM & Memory Profiling:** Measures resident set size (RSS via getrusage) alongside disk footprint to evaluate space/memory trade-offs.
- **Automated Unit Testing:** Integrated zero-dependency C++17 test suite (16 suites, 119 assertions) with native CMake/CTest support.

2. System Architecture & End-to-End Pipeline

The pipeline is organized into a modular layered architecture ensuring clear separation between raw ingestion (Datalake), queryable storage (Datamarts), and orchestration (Control Layer):

Component	Path & Artifacts	Function & Core Responsibilities
Datalake	data/datalake/	Stores raw headers and cleaned body text across Time, Book, or Batch folder hierarchies. Fast O(1) direct locator with fallback recursive scanner.
Metadata Mart	data/datamarts/metadata.db	SQLite database storing extracted header fields (title, author, language, timestamps, paths) indexed via B-trees.
Inverted Indexes	inverted_index.json inverted_index_hier/ inverted_index.bin	Three distinct storage formats supporting keyword lookup and Boolean AND/OR queries over tokenized vocabulary.
Control Layer	control/*.txt	State tracking queue (downloaded_books.txt, indexed_books.txt) orchestrating Gutenberg downloads and indexing atomically.
Test Harness	c++/tests/	Comprehensive automated test framework verifying all units and regression safety via ctest / make test.

3. Modular Codebase Breakdown

The codebase adheres to clean code principles with concise, highly focused modules:

- **Datalake (Datalake.hpp / Datalake.cpp):** Computes layout paths, saves documents, detects unindexed candidate books via single-pass directory scanning, and handles idempotent recovery.
- **MetadataExtractor (MetadataExtractor.hpp / MetadataExtractor.cpp):** Regex-based Gutenberg header parsing and transaction-wrapped SQLite CRUD operations.
- **Tokenizer (Tokenizer.hpp / Tokenizer.cpp):** Enforces the shared contract rule: lowercases text, extracts [A-Za-z]+ ASCII tokens, dropping numbers/punctuation without stemming or stopwords.

- **JsonIndex, HierarchicalIndex & BinaryIndex:** Implements monolithic JSON, folder sharding (A-Z/_), and custom BIDX binary storage with term dictionary headers and direct random seek postings access.
- **QueryEngine:** High-level query interface providing single-term lookups, Boolean AND (set intersection), and Boolean OR (set union).
- **ControlLayer:** State orchestrator managing the pipeline loop and Gutenberg downloads via libcurl.
- **BenchmarkRunner:** Automated multi-scale harness executing sweeps across scales (100, 1k, 10k, 100k) with RSS RAM profiling and detailed JSON logging.

4. Experimental Results & Multi-Scale Benchmark Analysis

Benchmarks were executed on macOS (Apple Silicon M-series) using the 20 authentic Project Gutenberg books mandated by shared/CONTRACT.md (comprising over 12 million characters of genuine prose).

4.1 Inverted Index Benchmark: Scalability, Memory (RAM) & Trade-offs

Evaluated across synthetic scales (100, 1,000, and 10,000 books). Query latency was measured across the 10 fixed contract search terms (time, love, heart, nothing, death, truth, fortune, monster, wonderland, darcy).

Structure	Scale	Build Time	Lookup avg	Update Cost	RAM (RSS)	Disk Footprint
json_monolithic	100	0.061 s	0.001 ms	396.05 ms	3.3 GB	8.8 MB
hierarchical	100	2.475 s	0.026 ms	0.30 ms	3.3 GB	4.7 MB
binary_compact	100	0.123 s	0.996 ms	186.64 ms	3.3 GB	3.4 MB
json_monolithic	1,000	0.391 s	0.001 ms	968.08 ms	3.3 GB	82.6 MB
hierarchical	1,000	3.058 s	0.077 ms	0.83 ms	3.3 GB	47.8 MB
binary_compact	1,000	0.206 s	0.998 ms	314.99 ms	3.3 GB	28.0 MB
json_monolithic	10,000	4.589 s	0.003 ms	10,318.06 ms (10.3 s)	3.3 GB	820.2 MB
hierarchical	10,000	7.167 s	0.895 ms	3.80 ms	3.3 GB	478.1 MB
binary_compact	10,000	1.235 s	1.385 ms	1,770.94 ms (1.7 s)	3.3 GB	273.9 MB

Index Structure Trade-offs & Analysis:

- **JSON Rewrite Bottleneck:** At 10,000 books, adding a book requires parsing and reserializing an 820 MB file, taking 10.31 seconds. This empirically confirms the unsuitability of monolithic text formats for continuous ingestion.
- **Hierarchical Incremental Superiority:** Updating the hierarchical index takes only 3.80 ms at 10,000 books—over 2,700x faster than JSON—since only the exact term files contained in the new book are opened and appended to.
- **Binary Space & Build Efficiency:** The binary compact format (BIDX) requires only 273.9 MB (66% less disk space than JSON) and builds in 1.23 seconds at 10,000 scale.

4.2 Metadata Storage Benchmark (SQLite Ingestion & B-Tree Query Stress Test)

Measures batch insertion speed and response times for find_by_id (primary key B-tree) and find_by_author comparing a repeated author (Charles Dickens, 30% frequency) against a unique author (Lewis Carroll, 10% frequency).

Scale	Insert Time	Throughput	ID Lookup	Dickens Query	Carroll Query	DB Size
100	0.0084 s	11,845 rows/s	0.077 ms	0.094 ms	0.095 ms	28 KB
1,000	0.0031 s	321,435 rows/s	0.078 ms	0.222 ms	0.184 ms	168 KB
10,000	0.0172 s	580,523 rows/s	0.077 ms	1.367 ms	0.950 ms	1.56 MB

4.3 Datalake Partitioning Benchmark (Multi-Scale Sweeps: 100, 1,000, 10,000)

Evaluates the three datalake storage hierarchies across write throughput, lookup latency, incremental detection cost, and idempotent recovery behavior after simulated document loss.

Layout	Scale	Write Time	Throughput	Lookup avg	Incr Detect	Incr Write	Recovery	Dirs
time_based	100	0.081 s	1,229 b/s	0.004 ms	0.000 s	0.009 s	0.008 s	2
book_based	100	0.087 s	1,143 b/s	0.004 ms	0.000 s	0.011 s	0.008 s	100
batch_based	100	0.086 s	1,160 b/s	0.004 ms	0.000 s	0.011 s	0.009 s	1
time_based	1,000	0.914 s	1,094 b/s	0.006 ms	0.001 s	0.096 s	0.113 s	2
book_based	1,000	0.972 s	1,028 b/s	0.005 ms	0.001 s	0.093 s	0.098 s	1,000
batch_based	1,000	0.958 s	1,044 b/s	0.004 ms	0.001 s	0.087 s	0.095 s	2
time_based	10,000	9.347 s	1,069 b/s	0.005 ms	0.011 s	0.929 s	2.024 s	2
book_based	10,000	11.503 s	869 b/s	0.008 ms	0.015 s	1.079 s	1.397 s	10,000
batch_based	10,000	10.721 s	932 b/s	0.004 ms	0.012 s	1.032 s	1.347 s	20

Datalake Hierarchy Trade-offs:

- **Time-Based (YYYYMMDD/HH/)**: Achieves highest write throughput (1,069 books/s at 10k scale) and minimizes directory inode pressure (only 2 folders created), making it the recommended architecture for continuous temporal ingestion pipelines.
- **Batch-Based (batch_N/, 500 books/dir)**: Provides optimal balance between directory depth and bounded file counts (20 folders at 10,000 scale), maintaining sub-0.005 ms lookup latency.
- **Book-Based (ID/)**: Degrades at scale due to creating 10,000 distinct directory inodes, resulting in 20% lower ingestion throughput.
- **Incremental Processing & Recovery**: Single-pass set detection identifies unindexed books in 11 milliseconds for 6,000 candidates, and idempotent recovery repairs 1,000 lost documents in under 2 seconds.

5. Automated Testing Suite & Verification Guide

The C++ module includes a native automated unit testing harness integrated with CMake and CTest. The test suite comprises **16 distinct test suites and 119 verified assertions** covering all functional units with 100% pass rate:

Test Suite Name	Target Module	Assertions & Validations
Tokenizer_BasicExtraction	Tokenizer	Verifies sequential word extraction and non-alpha character stripping.
Tokenizer_ContractCompliance	Tokenizer	Validates cross-language contract: lowercase, [A-Za-z]+, no stopword removal.
Metadata_HeaderParsing	MetadataExtractor	Tests regex extraction of Title, Author, Language on genuine Gutenberg headers.
Metadata_SQLiteCRUD	MetadataExtractor	Validates SQLite insertion, primary key queries, and author filtering.
Metadata_BatchInsert	MetadataExtractor	Tests high-speed transactional batch insertions into SQLite.
Datalake_PathGeneration	Datalake	Verifies path resolution across TimeBased, BookBased, and BatchBased layouts.
Datalake_IncrementalAndRecovery	Datalake	Validates unindexed book detection and idempotent recovery after simulated loss.
Indices_ConsistencyAcrossStructures	Inverted Indexes	Confirms JSON, Hierarchical, and Binary return identical postings across vocabulary.
QueryEngine_BooleanSearch	QueryEngine	Tests multi-term Boolean AND (intersection) and Boolean OR (union) resolution.
Control_PipelineOrchestration	ControlLayer	End-to-end test of ingestion, state tracking, and datamart updates.

Reproducible Build & CLI Execution Guide:

```
# 1. Build executables (search_engine and run_tests) cd c++ && cmake -B build -S . && cmake --build build # 2. Run automated test suite (CTest or Direct) ctest --test-dir build --output-on-failure ./build/run_tests # 3. Run multi-scale benchmarks (Datalake, Inverted Index, Metadata) ./build/search_engine bench-all 100,1000,10000 # 4. Sample dataset ingestion & Inverted Index Queries ./build/search_engine test-sample ./build/search_engine query truth all ./build/search_engine query-and truth fortune ./build/search_engine query-or wonderland darcy
```